JAVA SWING & OOP PROJECT

Personal Expense Tracker

An Object-Oriented Desktop Financial Management Application

SYSTEM ARCHITECTURE

Architectural Overview

Combining Clean OOP Principles with Java Swing Desktop Interfaces

Core OOP Principles

Encapsulation & Abstraction

Private Fields & Contract: The `Transaction` abstract class encapsulates core fields like `amount`, `date`, and `description`, protecting them behind public getter and setter accessors.

Abstract Methods: Enforces standard behavior by declaring `calculateImpact()` without concrete implementation details at the top level.

Inheritance & Polymorphism

Class Hierarchy: Concrete subclasses `Income` and `Expense` extend `Transaction`, inheriting attributes while adding tailored behaviors.

Polymorphic Calculations: Dynamic calculation of net account balance via `calculateImpact()` without checking concrete instance types explicitly.

Key System Features

- **Interactive Java Swing GUI:** Provides a desktop dashboard built with `JFrame`, `GridLayout`, and `JTable` for transaction management.
- **Real-Time Balance Updates:** Dynamically recalculates net account balance using polymorphic evaluation across all logged income and expense items.
- **Data Export Interface:** Implements the `Exportable` interface across transaction types to generate standardized CSV output rows.
- **Exception & Error Handling:** Validates user numerical inputs with `NumberFormatException` popups and catches I/O errors during file writes.

Interactive GUI Dashboard

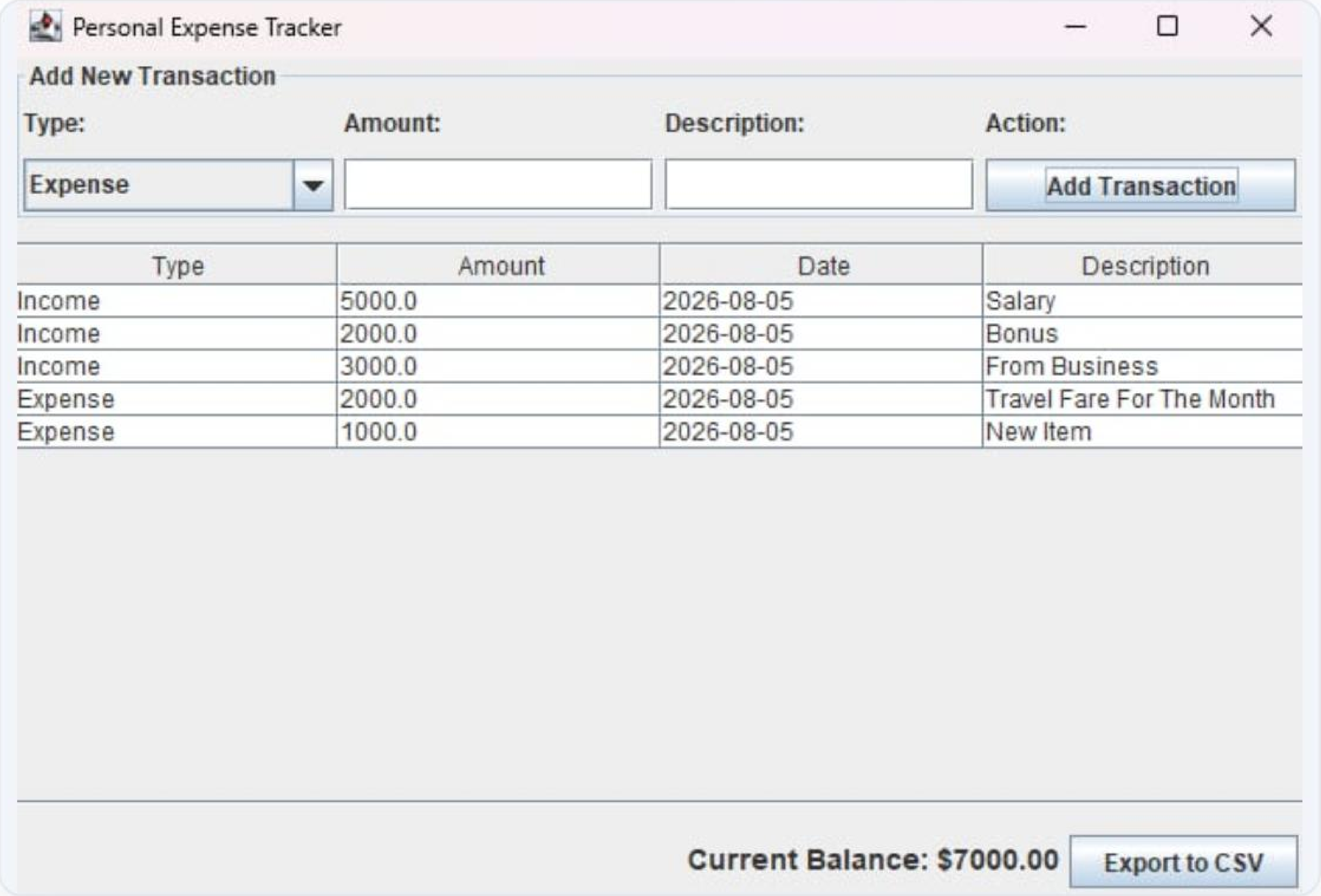
User Interface Layout

The application presents a clean desktop UI designed with Java Swing components:

Top Input Panel: Form inputs utilizing `JComboBox` for transaction type, `JTextField` for amounts and descriptions, and action trigger buttons.

Center Data View: A dynamic `JTable` backed by a `DefaultTableModel` wrapped inside a `JScrollPane` for full visibility.

Bottom Control Panel: Features real-time balance displays and the `Export to CSV` trigger button.



Class Structure & Roles

Class / Interface	Type	Key Responsibility	OOP Role
`Exportable`	Interface	Defines `generateCSV()` contract for data persistence	Interface Realization
`Transaction`	Abstract Class	Encapsulates common data & abstract `calculateImpact()`	Abstraction & Encapsulation
`Income`	Concrete Subclass	Returns positive impact (+amount) and CSV formatting	Inheritance & Polymorphism
`Expense`	Concrete Subclass	Returns negative impact (-amount) and CSV formatting	Inheritance & Polymorphism
`PersonalExpenseTrackerApp`	Swing Frame	Manages GUI, event listeners, list state, and CSV File I/O	Application Controller

Technical Stack

Java Swing & AWT

Built using standard Java desktop UI frameworks including `JFrame`, `JTable`, `DefaultTableModel`, and event-driven `ActionListener` delegates.

Java Collections API

Utilizes `List` and `ArrayList` to maintain an in-memory collection of polymorphic transaction instances efficiently.

File I/O & CSV Writer

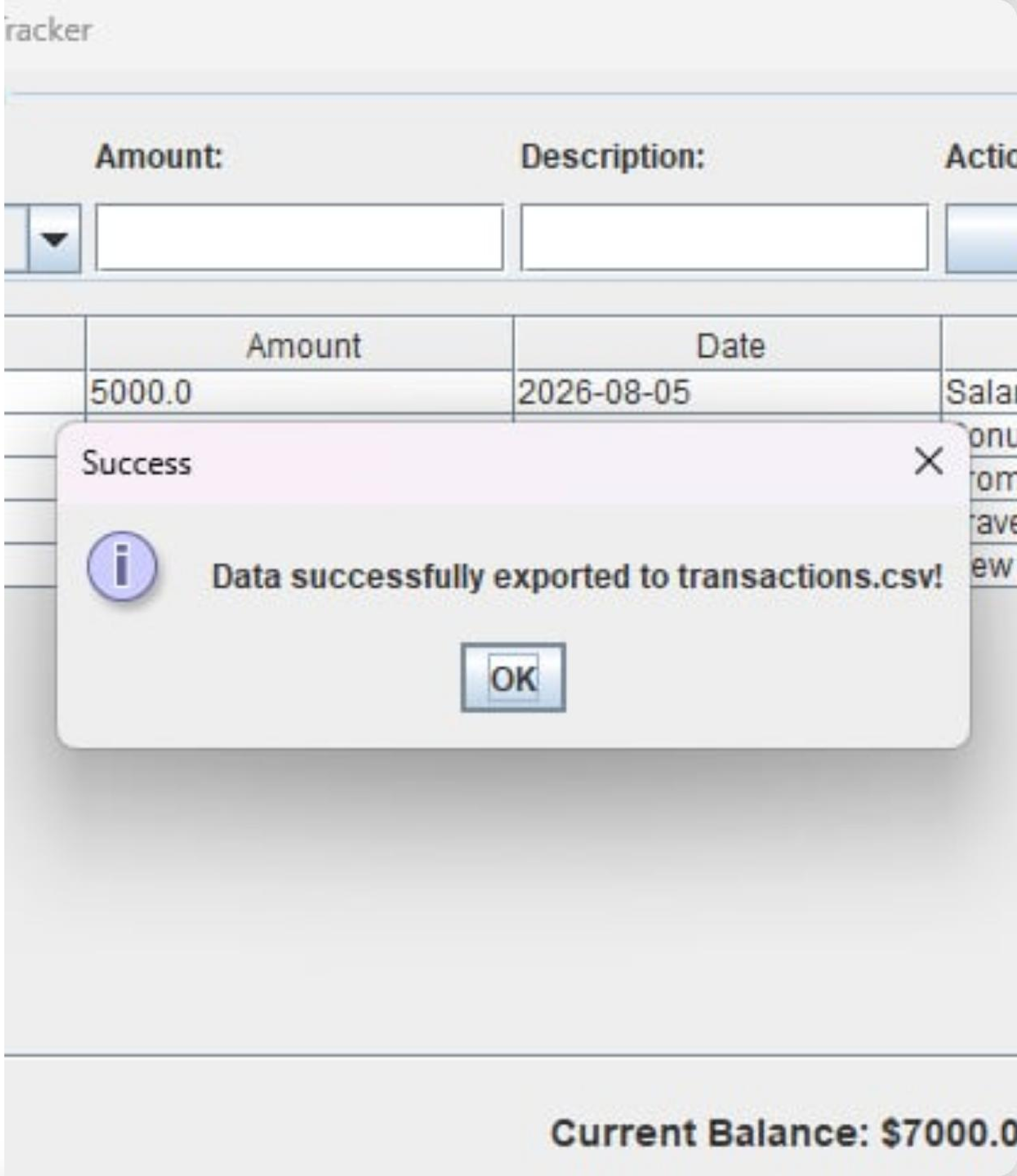
Leverages Java `FileWriter` and try-with-resources blocks to safely stream structured data records into disk files (e.g., `transactions.csv`).

CSV Data Export

Persistent Storage

The application enables easy persistence by exporting current transaction logs directly into CSV format.

Upon clicking **Export to CSV**, the app iterates polymorphically over all transactions, calling `generateCSV()` and writing formatted strings to disk with user feedback popups.



Project Highlights

100%

Pure Java & OOP

Extensible & Maintainable

By strictly enforcing object-oriented separation of concerns, the Personal Expense Tracker can be easily extended with new transaction types (e.g., `Investment`, `Transfer`) without altering existing UI or calculations.

The implementation cleanly bridges low-level OOP principles with practical GUI development and file operations.

Questions & Discussion

Thank you for reviewing the Personal Expense Tracker Project.

Java Desktop Application • OOP Architecture Demo